JAVA 300-PROBLEM — INDIVIDUAL CHECKLIST

Corrected version: every problem has its own implementation requirements and desired-output checklist

Use this with the original 300-problem PDF. Each problem below has its own checklist. The checklist identifies the specific classes, fields, methods, constructors, blocks, objects, calculations, traces, and output that must be completed for that particular problem.

LEVEL 1 — BEGINNER

Class & Object Use

Problem 1
Problem: Create a Student class containing name, age, roll number, and course. Create an object and display the student's details.
What you must complete:
■ Create the required classes: Student, Course.
■ Declare the name field.
■ Declare the age field.
■ Declare the roll number field.
■ Declare the course field.
■ Implement a display method/output section for the requested details.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the object details, count/number required by the question.

Problem 2
Problem: Create an Employee class containing employee ID, name, department, and salary. Create three employee objects and display their details.
What you must complete:
■ Create the required classes: Employee, Department, Salary.
■ Declare the employee ID field.
■ Declare the name field.
■ Declare the department field.
■ Declare the salary field.
■ Implement a display method/output section for the requested details.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the object details, calculated amount required by the question.

Problem 3
Problem: Create a MobilePhone class containing brand, model, RAM, storage, and price. Create two phone objects and display them.
What you must complete:
■ Create the MobilePhone class.
■ Declare the brand field.
■ Declare the model field.
■ Declare the RAM field.
■ Declare the storage field.
■ Declare the price field.
■ Implement a display method/output section for the requested details.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 4
Problem: Create a BankAccount class containing account number, holder name, and balance. Create an account object and display its information.
What you must complete:
■ Create the required classes: BankAccount, Bank.
■ Declare the account number field.
■ Declare the holder name field.
■ Declare the balance field.
■ Implement a display method/output section for the requested details.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the object details, count/number required by the question.

Problem 5
Problem: Create a Product class containing product name, price, and quantity. Create an object and calculate its total value.

What you must complete:
■ Create the Product class.
■ Declare the product name field.
■ Declare the price field.
■ Declare the quantity field.
■ Implement the requested calculation using the object's data.
■ Compute the requested total.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount, total required by the question.

Problem 6
Problem: Create a Book class containing title, author, ISBN, and price. Create three book objects and display their information.
What you must complete:
■ Create the Book class.
■ Declare the title field.
■ Declare the author field.
■ Declare the ISBN field.
■ Declare the price field.
■ Implement a display method/output section for the requested details.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the object details, calculated amount required by the question.

Problem 7
Problem: Create a Car class containing brand, model, color, and price. Create two cars and display their details.
What you must complete:
■ Create the Car class.
■ Declare the brand field.
■ Declare the model field.
■ Declare the color field.
■ Declare the price field.
■ Implement a display method/output section for the requested details.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the object details, calculated amount required by the question.

Problem 8
Problem: Create a Movie class containing title, genre, duration, and rating. Create three movie objects and display the movie with the highest rating.
What you must complete:
■ Create the Movie class.
■ Declare the title field.
■ Declare the genre field.
■ Declare the duration field.
■ Declare the rating field.
■ Implement a display method/output section for the requested details.
■ Create the required objects and compare the requested numeric/property value.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 9
Problem: Create a Rectangle class containing length and width. Create two objects and determine which rectangle has the larger area.
What you must complete:
■ Create the Rectangle class.
■ Declare the length field.
■ Declare the width field.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 10
Problem: Create a Student class containing marks in Mathematics, Physics, and Chemistry. Calculate total and average marks.
What you must complete:
■ Create the Student class.
■ Declare the age field.
■ Declare the required subject-mark fields.
■ Implement the requested calculation using the object's data.
■ Compute the average from the required values.

■ Compute the requested total.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount, average, total required by the question.

Problem 11
Problem: Create an Employee class containing basic salary. Calculate annual salary using an object method.
What you must complete:
■ Create the required classes: Employee, Salary.
■ Declare the salary field.
■ Declare a basic-salary field.
■ Implement the method(s) that perform the requested action.
■ Implement the requested calculation using the object's data.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount required by the question.

Problem 12
Problem: Create a ShoppingItem class containing name, price, and quantity. Calculate the amount payable for the item.
What you must complete:
■ Create the ShoppingItem class.
■ Declare the name field.
■ Declare the price field.
■ Declare the quantity field.
■ Implement the requested calculation using the object's data.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount required by the question.

Problem 13
Problem: Create a Temperature class storing Celsius temperature. Add methods to convert it to Fahrenheit and Kelvin.
What you must complete:
■ Create the Temperature class.
■ Declare the Celsius temperature field.
■ Implement the method(s) that perform the requested action.
■ Implement the requested unit/temperature conversions.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 14
Problem: Create a Consumer class containing consumer number, name, and units consumed. Calculate an electricity bill using a simple per-unit rate.
What you must complete:
■ Create the Consumer class.
■ Declare the consumer number field.
■ Declare the name field.
■ Declare the units consumed field.
■ Implement the requested calculation using the object's data.
■ Calculate the bill from units consumed and the stated rate/rule.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount, count/number required by the question.

Problem 15
Problem: Create an ATMAccount class containing account number and balance. Add methods for deposit, withdrawal, and displaying balance.
What you must complete:
■ Create the ATMAccount class.
■ Declare the account number field.
■ Declare the balance field.
■ Implement the method(s) that perform the requested action.
■ Implement a display method/output section for the requested details.
■ Implement deposit and update the balance.
■ Implement withdrawal and update the balance.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

this Keyword

Problem 16

Problem: Create a Student constructor where parameters have the same names as instance variables. Use this to initialize the student's name, age, and course.

What you must complete:

■ Create the required classes: Student, Course.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Treat this as the current object and compare its required fields with the supplied object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the comparison result required by the question.

Problem 17

Problem: Create an Employee class whose constructor receives employee ID, name, department, and salary. Use this to initialize all fields.

What you must complete:

■ Create the required classes: Employee, Department, Salary.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the calculated amount required by the question.

Problem 18

Problem: Create a Product class with a constructor accepting product name, price, and quantity. Use this.

What you must complete:

■ Create the Product class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the calculated amount required by the question.

Problem 19

Problem: Create a BankAccount class where the constructor parameter balance has the same name as the instance variable. Correctly initialize it using this.

What you must complete:

■ Create the required classes: BankAccount, Bank.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Treat this as the current object and compare its required fields with the supplied object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the comparison result, count/number required by the question.

Problem 20

Problem: Create a MobilePhone class with a setDetails() method. Use this to assign brand, model, and price.

What you must complete:

■ Create the MobilePhone class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Implement the requested setter methods and use this.field = parameter.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the object details, calculated amount required by the question.

Problem 21

Problem: Create a Customer class with setName(), setPhone(), and setAddress() methods. Use this.

What you must complete:

■ Create the required classes: Customer, Address.
■ Declare the instance fields that the constructor/setter/method must update.
■ Implement the requested setter methods and use this.field = parameter.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 22

Problem: Create an Employee class with a changeSalary() method that updates the current employee's salary using this.

What you must complete:

■ Create the required classes: Employee, Salary.
■ Declare the instance fields that the constructor/setter/method must update.
■ Create the required object(s) and call the methods in the sequence required by the problem.

■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the calculated amount required by the question.

Problem 23
Problem: Create a Student class with a method isSameStudent(Student student). Use this to compare the current student with another student.
What you must complete:
■ Create the Student class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Treat this as the current object and compare its required fields with the supplied object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the comparison result required by the question.

Problem 24
Problem: Create a BankAccount class with transfer(BankAccount target, double amount). Use this to represent the account from which money is transferred.
What you must complete:
■ Create the required classes: BankAccount, Bank.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use this as the source/current account and update both source and destination balances.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the count/number required by the question.

Problem 25
Problem: Create a Student class whose setName() method returns this. Use it to chain method calls.
What you must complete:
■ Create the Student class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Implement the requested setter methods and use this.field = parameter.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 26
Problem: Create a Customer class where setName(), setAge(), and setCity() each return this. Create a customer using method chaining.
What you must complete:
■ Create the Customer class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Implement the requested setter methods and use this.field = parameter.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 27
Problem: Create a Cart class where addItem() returns this. Add several products using chained calls.
What you must complete:
■ Create the required classes: Product, Car, Cart.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 28
Problem: Create an Employee method that compares the current employee's salary with another employee's salary using this.
What you must complete:
■ Create the required classes: Employee, Salary.
■ Declare the instance fields that the constructor/setter/method must update.
■ Treat this as the current object and compare its required fields with the supplied object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the calculated amount, comparison result required by the question.

Problem 29
Problem: Create a Person class with multiple constructors. Use this() so one constructor calls another.
What you must complete:
■ Create the Person class.

■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Create the overloaded constructors and use this(...) as the first statement to delegate initialization.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

static Keyword

Problem 30
Problem: Create a Student class where collegeName is static. Create five students and demonstrate that they share the same college name.
What you must complete:
■ Create the required classes: Student, College.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the comparison result required by the question.

Problem 31
Problem: Create a static variable that counts how many Student objects have been created.
What you must complete:
■ Create the Student class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 32
Problem: Create an Employee class with a static companyName. Create multiple employees.
What you must complete:
■ Create the required classes: Employee, Company.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 33
Problem: Create a BankAccount class with a static interest rate shared by all accounts.
What you must complete:
■ Create the required classes: BankAccount, Bank.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Demonstrate that all objects read the same static/shared value.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 34
Problem: Create a Product class with a static GST rate. Calculate tax for different products.
What you must complete:
■ Create the required classes: Product, Tax, GST.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the calculated amount required by the question.

Problem 35
Problem: Create a static counter that automatically generates employee IDs.
What you must complete:
■ Create the Employee class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

■ Desired output: clearly show the count/number required by the question.

Problem 36
Problem: Create an Order class where every newly created order receives a unique order number using a static variable.
What you must complete:
■ Create the Order class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 37
Problem: Create a class with a static method that converts kilometers to miles.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Implement the requested static method and call it without requiring an instance when appropriate.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 38
Problem: Create a ElectricityBill class with a static per-unit electricity rate.
What you must complete:
■ Create the ElectricityBill class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the calculated amount required by the question.

Problem 39
Problem: Create a School class where school name and school address are static and shared by all students.
What you must complete:
■ Create the required classes: Student, School, Address.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Demonstrate that all objects read the same static/shared value.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 40
Problem: Create a static variable that counts the number of products added across all shopping cart objects.
What you must complete:
■ Create the required classes: Product, Car, Cart.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 41
Problem: Create a static transaction counter shared by all bank accounts.
What you must complete:
■ Create the required classes: Transaction, Bank.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Demonstrate that all objects read the same static/shared value.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 42
Problem: Create a Patient class where a static variable tracks the total number of registered patients.
What you must complete:
■ Create the Patient class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.

■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the total, count/number required by the question.

Problem 43
Problem: Create a program where a company name is static but employee name and salary are instance variables. Demonstrate the difference.
What you must complete:
■ Create the required classes: Employee, Company, Salary.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Demonstrate that all objects read the same static/shared value.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the calculated amount required by the question.

Constructors

Problem 44
Problem: Create a Student class with a constructor that initializes name, age, and roll number.
What you must complete:
■ Create the Student class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the count/number required by the question.

Problem 45
Problem: Create an Employee class whose constructor initializes employee details.
What you must complete:
■ Create the Employee class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the object details required by the question.

Problem 46
Problem: Create a Product class whose constructor initializes name, price, and quantity.
What you must complete:
■ Create the Product class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the calculated amount required by the question.

Problem 47
Problem: Create a BankAccount class using a parameterized constructor.
What you must complete:
■ Create the required classes: BankAccount, Bank.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the count/number required by the question.

Problem 48
Problem: Create a Car class with a constructor accepting brand, model, and price.
What you must complete:
■ Create the Car class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the calculated amount required by the question.

Problem 49
Problem: Create a Book class with a constructor accepting title, author, and price.
What you must complete:
■ Create the Book class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.

■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the calculated amount required by the question.

Problem 50
Problem: Create a Customer class with a no-argument constructor that assigns default values.
What you must complete:
■ Create the Customer class.
■ Provide a no-argument/default constructor with the specified default values.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 51
Problem: Create a Student class with constructors allowing a student to be created using name only, name + age, and name + age + course.
What you must complete:
■ Create the required classes: Student, Course.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 52
Problem: Create overloaded constructors for a Product: default product; name + price; name + price + quantity.
What you must complete:
■ Create the Product class.
■ Provide a no-argument/default constructor with the specified default values.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement each required overloaded constructor with a distinct parameter list.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the calculated amount required by the question.

Problem 53
Problem: Create constructors for a Rectangle: default rectangle, square, and rectangle with length and width.
What you must complete:
■ Create the Rectangle class.
■ Provide a no-argument/default constructor with the specified default values.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Provide the square constructor and initialize both dimensions consistently.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 54
Problem: Create constructors that allow a BankAccount to be opened with either zero balance or an initial deposit.
What you must complete:
■ Create the required classes: BankAccount, Bank.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the count/number required by the question.

Problem 55
Problem: Create overloaded constructors for a MobilePhone with different amounts of information.
What you must complete:
■ Create the MobilePhone class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement each required overloaded constructor with a distinct parameter list.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the object details required by the question.

Problem 56
Problem: Create an Employee constructor that assigns Unknown when no employee name is provided.
What you must complete:
■ Create the Employee class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 57
Problem: Create a BankAccount constructor that rejects a negative opening balance.
What you must complete:
■ Create the required classes: BankAccount, Bank.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Validate constructor arguments and reject/handle invalid values according to the requirement.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the count/number required by the question.

Code Blocks

Problem 58
Problem: Create a Hospital class with a static block that prints Hospital System Started.
What you must complete:
■ Create the Hospital class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 59
Problem: Create a Student class with an instance initialization block that prints Student Object Created.
What you must complete:
■ Create the Student class.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 60
Problem: Create a class with a static block, instance block, and constructor. Predict the order in which they execute.
What you must complete:
■ Create the Order class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 61
Problem: Use a static block to initialize the bank's interest rate.
What you must complete:
■ Create the Bank class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 62
Problem: Use an instance block to initialize a default product category.
What you must complete:
■ Create the Product class.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 63
Problem: Create three Student objects and determine how many times the instance initialization block executes.
What you must complete:
■ Create the Student class.
■ Place multiple blocks in the required source order so their execution order can be observed.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 64
Problem: Create several BankAccount objects and determine how many times the static block executes.
What you must complete:
■ Create the required classes: BankAccount, Bank.

■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the count/number required by the question.

Problem 65
Problem: Create three static blocks that initialize different parts of a banking system. Predict their execution order.
What you must complete:
■ Create the required classes: Order, Bank.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Place multiple blocks in the required source order so their execution order can be observed.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 66
Problem: Create three instance blocks in an Employee class and determine their execution order.
What you must complete:
■ Create the required classes: Employee, Order.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Place multiple blocks in the required source order so their execution order can be observed.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 67
Problem: Create a static block that loads college information before any student object is created.
What you must complete:
■ Create the required classes: Student, College.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the object details required by the question.

Problem 68
Problem: Create an ATM class with a static block that prints ATM System Ready and an instance block that prints ATM Session Created.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 69
Problem: Create a class containing an instance block and constructor. Predict which executes first.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 70
Problem: Create a class containing static block, instance block, and constructor. Create two objects and predict the complete output.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

■ Desired output: clearly show the execution order/output required by the question.

Problem 71

Problem: Create Bank, Customer, and BankAccount classes with static blocks and instance blocks. Trace which blocks execute when the program starts and objects are created.

What you must complete:

■ Create the required classes: BankAccount, Customer, Bank.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the count/number, execution order/output required by the question.

Scope

Problem 72

Problem: Create an instance variable name and a local variable name inside a method. Determine which one is accessible.

What you must complete:

■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 73

Problem: Create an instance variable salary and a method parameter with the same name. Resolve the conflict using this.

What you must complete:

■ Create the Salary class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Create the required shadowing and resolve the instance field with this.field where needed.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the calculated amount, comparison result, scope/access result required by the question.

Problem 74

Problem: Create a local variable inside an addItem() method and attempt to access it from another method. Understand why it fails.

What you must complete:

■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 75

Problem: Create a variable inside an if block and determine whether it can be accessed outside the block.

What you must complete:

■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 76

Problem: Create a for loop containing a variable i. Determine where i can be accessed.

What you must complete:

■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 77

Problem: Create variables inside if-else blocks and analyze their scope.

What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 78
Problem: Create an instance variable, method parameter, and local variable with the same name. Determine which variable is used at each point.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Create the required shadowing and resolve the instance field with this.field where needed.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the comparison result, scope/access result required by the question.

Problem 79
Problem: Create a local PIN variable inside a login method. Determine whether another method can access it.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 80
Problem: Create an instance variable balance and a local variable balance inside withdraw(). Use this to distinguish them.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 81
Problem: Create nested blocks with variables representing patient information and determine their accessibility.
What you must complete:
■ Create the Patient class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the object details, scope/access result required by the question.

Problem 82
Problem: Create variables inside a loop that processes menu items. Determine their scope.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 83
Problem: Create a class variable, method variable, and block variable all named roomNumber. Determine which one is accessed.
What you must complete:
■ Create the Room class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the count/number, scope/access result required by the question.

Problem 84
Problem: Given a program containing class, method, and block variables, identify compilation errors caused by incorrect scope.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Keep method-local variables/parameters inside their declaring method.
■ Identify and fix every use of a variable outside its legal scope.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 85
Problem: Create one program demonstrating class scope, instance variable scope, method parameter scope, local variable scope, block scope, and loop variable scope.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Stack & Heap

Problem 86
Problem: Create a Student object and identify the reference and object conceptually.
What you must complete:
■ Create the Student class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 87
Problem: Create five student objects and draw the conceptual stack/heap relationship.
What you must complete:
■ Create the Student class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 88
Problem: Create two bank accounts and determine how references and objects relate.
What you must complete:
■ Create the Bank class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the count/number, final reference/object state required by the question.

Problem 89
Problem: Pass an employee object to a method that changes its salary. Explain why the original object changes.
What you must complete:
■ Create the required classes: Employee, Salary.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the calculated amount, final reference/object state required by the question.

Problem 90
Problem: Create two references pointing to the same product object. Change the product price using one reference.

What you must complete:
■ Create the Product class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the calculated amount, comparison result, final reference/object state required by the question.

Problem 91
Problem: Create two product objects with the same values. Explain why they are still separate objects.
What you must complete:
■ Create the Product class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the comparison result, final reference/object state required by the question.

Problem 92
Problem: Create a customer object, set its reference to null, and explain what happens to the object.
What you must complete:
■ Create the Customer class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ After each reference is removed/null, check whether any other live reference can still reach the object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 93
Problem: Create a cart and three product objects. Draw how the references connect the objects.
What you must complete:
■ Create the required classes: Product, Car, Cart.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 94
Problem: Create a method that creates a Student object and returns it.
What you must complete:
■ Create the Student class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ After the method returns, remove its stack frame and keep the returned reference if one was returned.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 95
Problem: Pass one BankAccount object to a method and modify its balance. Trace the reference.
What you must complete:
■ Create the required classes: BankAccount, Bank.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Show the object field changing while the reference still points to the same heap object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the count/number, execution order/output, final reference/object state required by the question.

Problem 96
Problem: Pass an integer and a Student object to separate methods. Modify both and compare the results.
What you must complete:
■ Create the Student class.
■ List every local/parameter reference and every object created with new.

■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Show the object field changing while the reference still points to the same heap object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the comparison result, final reference/object state required by the question.

Problem 97
Problem: Create three references to one Book object. Set the references to null one by one and determine when the object becomes unreachable.
What you must complete:
■ Create the Book class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ After each reference is removed/null, check whether any other live reference can still reach the object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 98
Problem: Create an employee object inside a method and return it. Trace the reference after the method ends.
What you must complete:
■ Create the Employee class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ After the method returns, remove its stack frame and keep the returned reference if one was returned.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the execution order/output, final reference/object state required by the question.

Problem 99
Problem: Create a customer containing an address object. Draw the conceptual heap relationship.
What you must complete:
■ Create the required classes: Customer, Address.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 100
Problem: Create a small banking program containing Customer, BankAccount, and Transaction. Draw the objects and references and explain the stack/heap conceptually.
What you must complete:
■ Create the required classes: BankAccount, Customer, Transaction, Bank.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the count/number, final reference/object state required by the question.

LEVEL 2 — INTERMEDIATE

Class & Object Use

Problem 101
Problem: Create Student objects for a class and calculate the class average.
What you must complete:
■ Create the Student class.
■ Declare the age field.
■ Implement the requested calculation using the object's data.
■ Compute the average from the required values.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount, average required by the question.

Problem 102
Problem: Create multiple employees and calculate the total monthly salary paid by a company.
What you must complete:
■ Create the required classes: Employee, Company, Salary.
■ Declare the salary field.
■ Implement the requested calculation using the object's data.
■ Compute the requested total.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount, total required by the question.

Problem 103
Problem: Create multiple bank accounts and find the account with the highest balance.
What you must complete:
■ Create the Bank class.
■ Create the required objects and compare the requested numeric/property value.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the highest/largest matching object, count/number required by the question.

Problem 104
Problem: Create Book objects and allow a user to search for a book by title.
What you must complete:
■ Create the required classes: Book, User.
■ Declare the title field.
■ Search the objects using the specified field and report the matching result.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the matching search result required by the question.

Problem 105
Problem: Create Product objects and allow a customer to add multiple products to a cart.
What you must complete:
■ Create the required classes: Product, Car, Customer, Cart.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 106
Problem: Create Movie objects and determine which movie has the highest rating.
What you must complete:
■ Create the Movie class.
■ Declare the rating field.
■ Create the required objects and compare the requested numeric/property value.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the highest/largest matching object required by the question.

Problem 107
Problem: Create Room objects and identify available and occupied rooms.
What you must complete:
■ Create the required classes: Room, UPI.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 108
Problem: Create patient objects and search for a patient using patient ID.
What you must complete:
■ Create the Patient class.
■ Search the objects using the specified field and report the matching result.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the matching search result required by the question.

Problem 109
Problem: Create menu item objects and calculate the bill for several ordered items.
What you must complete:
■ Create the Order class.
■ Implement the requested calculation using the object's data.
■ Calculate the bill from units consumed and the stated rate/rule.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount required by the question.

Problem 110
Problem: Create multiple consumer objects and calculate each consumer's electricity bill.
What you must complete:
■ Create the Consumer class.
■ Implement the requested calculation using the object's data.
■ Calculate the bill from units consumed and the stated rate/rule.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount required by the question.

Problem 111
Problem: Create employee objects and find the highest-paid employee.
What you must complete:
■ Create the Employee class.
■ Create the required objects and compare the requested numeric/property value.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the highest/largest matching object required by the question.

Problem 112
Problem: Create Vehicle objects and calculate rental charges based on number of days.
What you must complete:
■ Create the required classes: Vehicle, Rental.
■ Implement the requested calculation using the object's data.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount, count/number required by the question.

Problem 113
Problem: Create multiple bank accounts and implement deposit and withdrawal operations.
What you must complete:
■ Create the Bank class.
■ Implement deposit and update the balance.
■ Implement withdrawal and update the balance.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the count/number required by the question.

Problem 114
Problem: Create a wallet object containing balance and implement add-money and payment methods.
What you must complete:
■ Create the required classes: Wallet, Payment.
■ Implement the method(s) that perform the requested action.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 115
Problem: Create passenger objects and calculate total ticket price based on age and travel class.
What you must complete:
■ Create the required classes: Passenger, Ticket.

■ Implement the requested calculation using the object's data.
■ Compute the requested total.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount, total required by the question.

this Keyword

Problem 116
Problem: Use this in a constructor to initialize all customer information.
What you must complete:
■ Create the Customer class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the object details required by the question.

Problem 117
Problem: Use this in setter methods for employee details.
What you must complete:
■ Create the Employee class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Implement the requested setter methods and use this.field = parameter.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the object details required by the question.

Problem 118
Problem: Use this to identify the source bank account in a transfer operation.
What you must complete:
■ Create the Bank class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use this as the source/current account and update both source and destination balances.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the count/number required by the question.

Problem 119
Problem: Compare the current product with another product using this.
What you must complete:
■ Create the Product class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Treat this as the current object and compare its required fields with the supplied object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the comparison result required by the question.

Problem 120
Problem: Create chained methods setName(), setAge(), setCourse(), and setMarks().
What you must complete:
■ Create the Course class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Implement the requested setter methods and use this.field = parameter.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 121
Problem: Use return this from addProduct() so multiple products can be added in one statement.
What you must complete:
■ Create the Product class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 122
Problem: Create an employee using chained setter calls.

What you must complete:
■ Create the Employee class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Implement the requested setter methods and use this.field = parameter.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 123
Problem: Create a Booking object whose methods return this to allow chained configuration.
What you must complete:
■ Create the required classes: Book, Booking.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 124
Problem: Create a Car object using chained methods for color, engine, price, and model.
What you must complete:
■ Create the Car class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the calculated amount required by the question.

Problem 125
Problem: Use this to compare two bank account objects.
What you must complete:
■ Create the Bank class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Treat this as the current object and compare its required fields with the supplied object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the comparison result, count/number required by the question.

Problem 126
Problem: Create a Student class with three constructors using this().
What you must complete:
■ Create the Student class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Create the overloaded constructors and use this(...) as the first statement to delegate initialization.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 127
Problem: Create overloaded constructors and make them call one central constructor.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 128
Problem: Create isSameCustomer(Customer c) using this.
What you must complete:
■ Create the Customer class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Treat this as the current object and compare its required fields with the supplied object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the comparison result required by the question.

Problem 129
Problem: Pass this from one object's method to another object's method and observe how the other object receives the current object.

What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the instance fields that the constructor/setter/method must update.
■ Pass this to the other object's method and use the received current-object reference.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

static Keyword

Problem 130
Problem: Track the total number of students registered using a static variable.
What you must complete:
■ Create the Student class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the total, count/number required by the question.

Problem 131
Problem: Generate unique employee IDs using a static counter.
What you must complete:
■ Create the Employee class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 132
Problem: Generate unique transaction IDs using a static variable.
What you must complete:
■ Create the Transaction class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 133
Problem: Every shopping order should automatically receive a unique ID.
What you must complete:
■ Create the Order class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 134
Problem: Automatically assign a unique patient number.
What you must complete:
■ Create the Patient class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 135
Problem: Generate unique consumer numbers using a static counter.
What you must complete:
■ Create the Consumer class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.

■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 136
Problem: Store the company name statically while employee details remain instance-specific.
What you must complete:
■ Create the required classes: Employee, Company.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Demonstrate that all objects read the same static/shared value.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the object details required by the question.

Problem 137
Problem: Store a common GST rate statically for all products.
What you must complete:
■ Create the required classes: Product, GST.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Demonstrate that all objects read the same static/shared value.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 138
Problem: Store a common interest rate statically and update it for all accounts.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Demonstrate that all objects read the same static/shared value.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 139
Problem: Create static methods for calculating GST, discount, and final price.
What you must complete:
■ Create the GST class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Implement the requested static method and call it without requiring an instance when appropriate.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the calculated amount, count/number required by the question.

Problem 140
Problem: Track how many transactions have been performed across all accounts.
What you must complete:
■ Create the Transaction class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 141
Problem: Track the total number of tickets sold across all movie booking objects.
What you must complete:
■ Create the required classes: Book, Movie, Booking, Ticket.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the total, count/number required by the question.

Problem 142
Problem: Create a static method that creates a BankAccount with a default configuration.
What you must complete:
■ Create the required classes: BankAccount, Bank.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Implement the requested static method and call it without requiring an instance when appropriate.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 143
Problem: Given a program containing static and instance members, identify which accesses are valid and which produce compilation errors.
What you must complete:
■ Create the Member class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Constructors

Problem 144
Problem: Create overloaded constructors for different amounts of student information.
What you must complete:
■ Create the Student class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement each required overloaded constructor with a distinct parameter list.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the object details required by the question.

Problem 145
Problem: Create constructors for employees with different salary configurations.
What you must complete:
■ Create the required classes: Employee, Salary.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create the constructor variants for the specified account/employee types.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the calculated amount required by the question.

Problem 146
Problem: Use constructor overloading for savings and current accounts.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create the constructor variants for the specified account/employee types.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the count/number required by the question.

Problem 147
Problem: Create products using different constructors depending on available information.
What you must complete:
■ Create the Product class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the object details required by the question.

Problem 148
Problem: Create constructors for standard, deluxe, and suite rooms.
What you must complete:
■ Create the Room class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 149
Problem: Create constructors for cars, bikes, and trucks with appropriate default values.
What you must complete:
■ Create the Car class.
■ Provide a no-argument/default constructor with the specified default values.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 150
Problem: Create overloaded constructors for movie information.
What you must complete:
■ Create the Movie class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement each required overloaded constructor with a distinct parameter list.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the object details required by the question.

Problem 151
Problem: Create a no-argument and parameterized customer constructor.
What you must complete:
■ Create the Customer class.
■ Provide a no-argument/default constructor with the specified default values.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 152
Problem: Create constructors that support an empty order and an order containing initial products.
What you must complete:
■ Create the required classes: Product, Order.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 153
Problem: Use constructor overloading for square and rectangular objects.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Provide the square constructor and initialize both dimensions consistently.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 154
Problem: Prevent creation of an account with an invalid opening balance.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Validate constructor arguments and reject/handle invalid values according to the requirement.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the count/number required by the question.

Problem 155
Problem: Create a copy-constructor-style constructor that creates a new student from an existing student.
What you must complete:
■ Create the Student class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement a copy-constructor-style constructor that copies the required fields from the source object.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 156
Problem: Create a new employee object using the information of another employee.
What you must complete:
■ Create the Employee class.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

■ Desired output: clearly show the object details required by the question.

Problem 157
Problem: Create overloaded constructors, this(), static blocks, and instance blocks. Predict the complete execution order.
What you must complete:
■ Create the Order class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement each required overloaded constructor with a distinct parameter list.
■ Use this(...) to route simpler constructors to the central initialization constructor.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the execution order/output required by the question.

Code Blocks

Problem 158
Problem: Use a static block to initialize bank-wide information.
What you must complete:
■ Create the Bank class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the object details required by the question.

Problem 159
Problem: Use an instance initialization block to assign a default student status.
What you must complete:
■ Create the Student class.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 160
Problem: Create multiple static blocks to initialize hospital configuration.
What you must complete:
■ Create the Hospital class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Place multiple blocks in the required source order so their execution order can be observed.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 161
Problem: Use static and instance blocks to distinguish ATM startup from ATM session creation.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 162
Problem: Trace static block → instance block → constructor execution.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 163
Problem: Determine how many times the instance block executes when five employees are created.
What you must complete:
■ Create the Employee class.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 164
Problem: Create classes with static initialization and determine which static blocks execute first.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 165
Problem: Use an instance block to initialize default room status.
What you must complete:
■ Create the Room class.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 166
Problem: Use a static block to initialize GST and an instance block to initialize product status.
What you must complete:
■ Create the required classes: Product, GST.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 167
Problem: Use an instance block to initialize the cart's item count.
What you must complete:
■ Create the required classes: Car, Cart.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the count/number required by the question.

Problem 168
Problem: Use an instance block to assign a default patient status.
What you must complete:
■ Create the Patient class.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 169
Problem: Create two classes where one class uses a static field from another class. Predict initialization order.
What you must complete:
■ Create the Order class.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 170
Problem: Create multiple constructors and instance blocks and trace the exact output.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Place multiple blocks in the required source order so their execution order can be observed.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 171
Problem: Create Bank, Customer, and Account classes with static blocks, instance blocks, and constructors. Predict the entire startup and object-creation sequence.
What you must complete:
■ Create the required classes: Customer, Bank.

■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the count/number, execution order/output required by the question.

Scope

Problem 172
Problem: Demonstrate shadowing between instance variable, method parameter, and local variable.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Create the required shadowing and resolve the instance field with this.field where needed.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 173
Problem: Use this to distinguish an instance balance from a local balance.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 174
Problem: Determine the scope of a product variable created inside a loop.
What you must complete:
■ Create the Product class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 175
Problem: Create nested blocks and determine variable accessibility.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 176
Problem: Create variables inside if blocks and determine whether they are accessible outside.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 177
Problem: Create a local PIN inside a login method and explain why other methods cannot directly access it.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 178
Problem: Use the same variable names at class, method, and block levels. Predict output.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the comparison result, execution order/output, scope/access result required by the question.

Problem 179
Problem: Create menu variables inside a loop and analyze their scope.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 180
Problem: Create passenger information at different scopes and determine which variables are accessible.
What you must complete:
■ Create the Passenger class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the object details, scope/access result required by the question.

Problem 181
Problem: Create temporary calculation variables inside a billing method and identify their scope.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the calculated amount, scope/access result required by the question.

Problem 182
Problem: Create three nested blocks and determine which variables are visible at every level.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 183
Problem: Fix a program containing variables accessed outside their valid scopes.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 184
Problem: Create a program with instance variables and local variables having identical names and solve the ambiguity using this.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 185
Problem: Analyze a 30-line real-life program and identify the scope of every variable.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Stack & Heap

Problem 186
Problem: Create five students and draw their conceptual stack/heap relationships.
What you must complete:
■ Create the Student class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 187
Problem: Create three accounts and trace the references.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the count/number, execution order/output, final reference/object state required by the question.

Problem 188
Problem: Create a cart containing several product objects and draw the object graph.
What you must complete:
■ Create the required classes: Product, Car, Cart.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 189
Problem: Pass an employee object to multiple methods and trace the references in each stack frame.
What you must complete:
■ Create the Employee class.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the execution order/output, final reference/object state required by the question.

Problem 190
Problem: Pass source and destination accounts to a transfer method and draw the stack/heap state.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the count/number, final reference/object state required by the question.

Problem 191
Problem: Pass an account object to a method and reassign the parameter to another account. Determine whether the caller's reference changes.
What you must complete:
■ Create the class/classes named or implied by the problem.

■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Show the parameter/reference changing targets and separately show whether the caller's reference changes.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the count/number, final reference/object state required by the question.

Problem 192
Problem: Create a customer containing an address object and draw the heap structure.
What you must complete:
■ Create the required classes: Customer, Address.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 193
Problem: Create an order containing several product references and draw the object relationships.
What you must complete:
■ Create the required classes: Product, Order.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 194
Problem: Create a library containing book objects and determine which objects remain reachable after removing references.
What you must complete:
■ Create the required classes: Book, Library.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 195
Problem: Create a hotel containing rooms and bookings. Identify objects and references.
What you must complete:
■ Create the required classes: Book, Room, Booking, Hotel.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 196
Problem: Create two objects that reference each other and analyze garbage-collection eligibility.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ After each reference is removed/null, check whether any other live reference can still reach the object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 197
Problem: Create a static object reference and determine why the object may remain reachable even after local references disappear.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Include the static reference as a class-level/root reference when checking reachability.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.

■ Desired output: clearly show the final reference/object state required by the question.

Problem 198

Problem: Create an object inside a method and return it. Trace the reference before and after the method returns.

What you must complete:

■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ After the method returns, remove its stack frame and keep the returned reference if one was returned.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the execution order/output, final reference/object state required by the question.

Problem 199

Problem: Create three methods that call one another while passing objects. Draw the stack frames.

What you must complete:

■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 200

Problem: Build a shopping system with Customer → Cart → Order → Product. Draw the conceptual stack and heap and explain which objects remain reachable after the customer logs out.

What you must complete:

■ Create the required classes: Product, Car, Customer, Cart, Order.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

LEVEL 3 — ADVANCED

Class & Object Use

Problem 201
Problem: Create Bank, Customer, BankAccount, and Transaction classes. Allow customers to deposit, withdraw, and transfer money.
What you must complete:
■ Create the required classes: BankAccount, Customer, Transaction, Bank.
■ Implement deposit and update the balance.
■ Implement withdrawal and update the balance.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the count/number required by the question.

Problem 202
Problem: Create User, BankAccount, UPIAccount, and Transaction classes. Implement a simplified UPI payment.
What you must complete:
■ Create the required classes: BankAccount, Transaction, Bank, UPIAccount, User, Payment, UPI.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the count/number required by the question.

Problem 203
Problem: Create Customer, Product, Cart, Order, and Payment classes for an Amazon-style shopping system.
What you must complete:
■ Create the required classes: Product, Car, Customer, Cart, Order, Payment.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 204
Problem: Create a Flipkart-style product system allowing customers to browse products, add them to carts, and place orders.
What you must complete:
■ Create the required classes: Product, Car, Customer, Cart, Order.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 205
Problem: Create passenger, train, seat, and ticket objects and implement basic railway ticket booking.
What you must complete:
■ Create the required classes: Book, Passenger, Booking, Train, Seat, Ticket.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 206
Problem: Create movie, theater, screen, seat, customer, and booking objects for a movie-booking system.
What you must complete:
■ Create the required classes: Book, Movie, Customer, Booking, Seat, Theater, Screen.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 207
Problem: Create restaurant, menu item, customer, order, delivery partner, and payment objects for a food-delivery system.
What you must complete:
■ Create the required classes: Customer, Order, Restaurant, Payment.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 208
Problem: Create patient, doctor, appointment, prescription, and hospital objects for hospital management.
What you must complete:
■ Create the required classes: Hospital, Patient, Doctor, Appointment, Prescription.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 209
Problem: Create hotel, room, customer, booking, and payment objects for hotel booking.
What you must complete:
■ Create the required classes: Book, Customer, Room, Booking, Payment, Hotel.
■ Create the required object(s) and supply test values.

■ Print the exact final value/details requested by the problem.

Problem 210
Problem: Create customer, vehicle, rental, and payment classes for vehicle rental.
What you must complete:
■ Create the required classes: Customer, Vehicle, Payment, Rental.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 211
Problem: Create parking lot, vehicle, parking slot, and parking ticket classes for a parking system.
What you must complete:
■ Create the required classes: Vehicle, Ticket.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 212
Problem: Create college, department, teacher, student, and course objects for college management.
What you must complete:
■ Create the required classes: Student, College, Department, Teacher, Course.
■ Declare the age field.
■ Declare the course field.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 213
Problem: Create library, member, book, issue record, and return record classes for library management.
What you must complete:
■ Create the required classes: Book, Library, Member.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

Problem 214
Problem: Create company, employee, department, salary, tax, and payroll objects for a payroll system.
What you must complete:
■ Create the required classes: Employee, Department, Company, Salary, Tax, Payroll.
■ Declare the department field.
■ Declare the salary field.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.
■ Desired output: clearly show the calculated amount required by the question.

Problem 215
Problem: Design a complete simplified e-commerce application using at least six interacting classes.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Create the required object(s) and supply test values.
■ Print the exact final value/details requested by the problem.

this Keyword

Problem 216
Problem: Create a fluent UPI account configuration using this and method chaining.
What you must complete:
■ Create the UPI class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the count/number required by the question.

Problem 217
Problem: Create chained methods for bank account holder, account type, branch, and opening balance.
What you must complete:
■ Create the Bank class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

■ Desired output: clearly show the count/number required by the question.

Problem 218
Problem: Use this to create a chained order-building system.
What you must complete:
■ Create the Order class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 219
Problem: Create a booking object using chained methods for room, date, customer, and payment.
What you must complete:
■ Create the required classes: Book, Customer, Room, Booking, Payment.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 220
Problem: Create chained methods for passenger name, age, train, seat, and class.
What you must complete:
■ Create the required classes: Passenger, Train, Seat.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 221
Problem: Create a food order using chained methods such as addItem(), setAddress(), and setPayment().
What you must complete:
■ Create the required classes: Order, Payment, Address.
■ Declare the instance fields that the constructor/setter/method must update.
■ Implement the requested setter methods and use this.field = parameter.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 222
Problem: Build an employee using chained setter methods.
What you must complete:
■ Create the Employee class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Implement the requested setter methods and use this.field = parameter.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 223
Problem: Create five overloaded constructors where all initialization eventually reaches one constructor using this().
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Create the overloaded constructors and use this(...) as the first statement to delegate initialization.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 224
Problem: Create a method that compares the current object with another object field by field.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the instance fields that the constructor/setter/method must update.
■ Treat this as the current object and compare its required fields with the supplied object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the comparison result required by the question.

Problem 225
Problem: Use this as the source account and another object as the destination account.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the instance fields that the constructor/setter/method must update.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the count/number required by the question.

Problem 226
Problem: Pass this from one object to another object's method and perform an operation.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the instance fields that the constructor/setter/method must update.
■ Pass this to the other object's method and use the received current-object reference.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 227
Problem: Allow cart.add(product1).add(product2).add(product3).checkout(); using this.
What you must complete:
■ Create the required classes: Product, Car, Cart.
■ Declare the instance fields that the constructor/setter/method must update.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 228
Problem: Create a complete employee object through chained methods returning this.
What you must complete:
■ Create the Employee class.
■ Declare the instance fields that the constructor/setter/method must update.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.

Problem 229
Problem: Analyze a program involving this, this(), overloaded constructors, method chaining, and multiple interacting objects. Predict the output and final object state.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the instance fields that the constructor/setter/method must update.
■ Use constructor parameters with the required names and initialize the matching fields with this.field.
■ Return this from each fluent method so subsequent calls operate on the same object.
■ Create the overloaded constructors and use this(...) as the first statement to delegate initialization.
■ Create the required object(s) and call the methods in the sequence required by the problem.
■ Print the requested final object state/result and verify that this referred to the intended current object.
■ Desired output: clearly show the execution order/output required by the question.

static Keyword

Problem 230
Problem: Maintain a transaction count shared by every bank account.
What you must complete:
■ Create the required classes: Transaction, Bank.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Demonstrate that all objects read the same static/shared value.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 231
Problem: Use static data to generate unique UPI transaction IDs.
What you must complete:
■ Create the required classes: Transaction, UPI.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.

■ Print the requested count, ID, shared value, or calculation result.

Problem 232
Problem: Generate unique railway PNR numbers using a static counter.
What you must complete:
■ Create the PNR class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 233
Problem: Generate unique patient IDs whenever a patient object is created.
What you must complete:
■ Create the Patient class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 234
Problem: Track total tickets booked across all theater objects.
What you must complete:
■ Create the required classes: Book, Ticket, Theater.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the total required by the question.

Problem 235
Problem: Generate unique order IDs across all customers.
What you must complete:
■ Create the required classes: Customer, Order.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 236
Problem: Track total employees created across departments.
What you must complete:
■ Create the required classes: Employee, Department.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the total required by the question.

Problem 237
Problem: Store the GST rate centrally and apply it to all products.
What you must complete:
■ Create the required classes: Product, GST.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 238
Problem: Update the bank interest rate and observe how every account uses the new rate.
What you must complete:
■ Create the Bank class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.

■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 239
Problem: Create a static collection/reference representing a shared transaction system.
What you must complete:
■ Create the Transaction class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Demonstrate that all objects read the same static/shared value.
■ Create the shared static collection/reference and add the required records to it.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 240
Problem: Create a static method that generates different account objects based on account type.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Create one static counter and update it at the exact event being counted.
■ Declare the common rate as static and use that one value in the calculation for every relevant object.
■ Implement the requested static method and call it without requiring an instance when appropriate.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the count/number required by the question.

Problem 241
Problem: Create a banking system where static initialization loads configuration before objects are created.
What you must complete:
■ Create the Bank class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Place class-wide setup in static initialization and ensure it occurs before the dependent use.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Problem 242
Problem: Create three classes with dependent static initialization and predict the order.
What you must complete:
■ Create the Order class.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Place class-wide setup in static initialization and ensure it occurs before the dependent use.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.
■ Desired output: clearly show the execution order/output required by the question.

Problem 243
Problem: Given a large program containing static fields, static methods, objects, constructors, and initialization blocks, identify what is shared, what belongs to individual objects, when static initialization occurs, and which objects remain reachable through static references.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare the per-object fields as instance members and the shared/class-wide value as static.
■ Use the static counter to generate the required unique ID/number for each new record/event.
■ Demonstrate that all objects read the same static/shared value.
■ Implement the requested static method and call it without requiring an instance when appropriate.
■ Place class-wide setup in static initialization and ensure it occurs before the dependent use.
■ Create the required objects/events to prove the static member is shared.
■ Print the requested count, ID, shared value, or calculation result.

Constructors

Problem 244
Problem: Create overloaded constructors for savings, current, and salary accounts.
What you must complete:
■ Create the Salary class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement each required overloaded constructor with a distinct parameter list.
■ Create the constructor variants for the specified account/employee types.
■ Create objects using each constructor that the problem asks you to demonstrate.

■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the calculated amount, count/number required by the question.

Problem 245
Problem: Create constructors supporting different account registration scenarios for a UPI account.
What you must complete:
■ Create the UPI class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the count/number required by the question.

Problem 246
Problem: Create product constructors supporting default, discounted, and premium products.
What you must complete:
■ Create the Product class.
■ Provide a no-argument/default constructor with the specified default values.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the count/number required by the question.

Problem 247
Problem: Create constructors for empty orders and orders containing initial products.
What you must complete:
■ Create the required classes: Product, Order.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 248
Problem: Create constructors for standard, deluxe, and suite hotel rooms.
What you must complete:
■ Create the required classes: Room, Hotel.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 249
Problem: Create railway ticket constructors supporting different passenger information configurations.
What you must complete:
■ Create the required classes: Passenger, Ticket.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the object details required by the question.

Problem 250
Problem: Create constructors for normal, premium, and IMAX movie tickets.
What you must complete:
■ Create the required classes: Movie, Ticket.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 251
Problem: Create overloaded constructors for permanent, temporary, and contract employees.
What you must complete:
■ Create the Employee class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement each required overloaded constructor with a distinct parameter list.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 252
Problem: Create constructors that initialize patient information and default status.
What you must complete:
■ Create the Patient class.
■ Provide a no-argument/default constructor with the specified default values.

■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the object details required by the question.

Problem 253
Problem: Create a copy-constructor-style design for creating a new customer from an existing customer.
What you must complete:
■ Create the Customer class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement a copy-constructor-style constructor that copies the required fields from the source object.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 254
Problem: Create a new bank account using another account's details while assigning a new account number.
What you must complete:
■ Create the Bank class.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the object details, count/number required by the question.

Problem 255
Problem: Create a Product constructor that rejects invalid price and quantity.
What you must complete:
■ Create the Product class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Validate constructor arguments and reject/handle invalid values according to the requirement.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the calculated amount required by the question.

Problem 256
Problem: Create a complete order system where overloaded constructors use this() to avoid duplicate initialization.
What you must complete:
■ Create the Order class.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement each required overloaded constructor with a distinct parameter list.
■ Use this(...) to route simpler constructors to the central initialization constructor.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.

Problem 257
Problem: Create a program containing overloaded constructors, constructor chaining, static blocks, instance blocks, and multiple objects. Predict the exact output and final field values.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Provide the parameterized constructor(s) with the exact parameter sets required.
■ Implement each required overloaded constructor with a distinct parameter list.
■ Use this(...) to route simpler constructors to the central initialization constructor.
■ Create objects using each constructor that the problem asks you to demonstrate.
■ Print the resulting object data and confirm each constructor initialized the correct fields.
■ Desired output: clearly show the execution order/output required by the question.

Code Blocks

Problem 258
Problem: Use static initialization blocks to load bank configuration.
What you must complete:
■ Create the Bank class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 259
Problem: Create multiple static blocks that initialize UPI payment configuration.
What you must complete:
■ Create the required classes: Payment, UPI.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.

■ Place multiple blocks in the required source order so their execution order can be observed.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 260
Problem: Initialize hospital configuration through static blocks.
What you must complete:
■ Create the Hospital class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 261
Problem: Initialize GST, delivery charges, and application configuration using static blocks.
What you must complete:
■ Create the GST class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 262
Problem: Use instance blocks to initialize default customer status.
What you must complete:
■ Create the Customer class.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 263
Problem: Use instance blocks to initialize default account state.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the count/number required by the question.

Problem 264
Problem: Use instance blocks to initialize room availability.
What you must complete:
■ Create the Room class.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 265
Problem: Use instance blocks to initialize employee status before the constructor runs.
What you must complete:
■ Create the Employee class.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 266
Problem: Create parent and child classes containing static blocks, instance blocks, and constructors. Predict execution order.
What you must complete:
■ Create the Order class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Create the parent and child classes and include the requested blocks/constructors in both.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 267
Problem: Create three classes where creating one object causes other classes to initialize.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Place multiple blocks in the required source order so their execution order can be observed.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Problem 268
Problem: Create an object inside a static initialization block and trace all resulting initialization.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 269
Problem: Create another object inside an instance block and trace the execution.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 270
Problem: Combine static field initialization, static blocks, instance field initialization, instance blocks, constructor chaining, and constructors. Predict exact output.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.
■ Desired output: clearly show the execution order/output required by the question.

Problem 271
Problem: Create a three-class banking application where object creation triggers a complicated sequence of static blocks, instance blocks, and constructors. Produce the exact execution timeline.
What you must complete:
■ Create the Bank class.
■ Add the requested static initialization block(s) and put the class-wide initialization/print statement inside them.
■ Add the requested instance initialization block(s) and put per-object initialization/print statements inside them.
■ Include the constructor(s) needed so their execution can be compared with the initialization blocks.
■ Place multiple blocks in the required source order so their execution order can be observed.
■ Trigger the required class initialization and create the specified number of objects.
■ Add clear print statements to every execution stage so the order is visible.
■ Write down the expected execution sequence and compare it with the actual output.

Scope

Problem 272
Problem: Create a program containing bank-level static variables, account-level instance variables, method variables, and block variables.
What you must complete:
■ Create the Bank class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the count/number, scope/access result required by the question.

Problem 273
Problem: Analyze the scope of transaction variables inside nested payment-processing blocks.
What you must complete:

■ Create the required classes: Transaction, Payment.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 274
Problem: Create nested loops for products and orders and determine variable accessibility.
What you must complete:
■ Create the required classes: Product, Order.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 275
Problem: Create passenger information at different scopes and determine which variables are accessible.
What you must complete:
■ Create the Passenger class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the object details, scope/access result required by the question.

Problem 276
Problem: Use the same variable names at class, method, and block levels in a hospital program. Predict every reference.
What you must complete:
■ Create the Hospital class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the comparison result, execution order/output, scope/access result required by the question.

Problem 277
Problem: Create room variables at multiple scope levels and resolve them.
What you must complete:
■ Create the Room class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 278
Problem: Analyze salary variables declared at class, method, loop, and conditional levels.
What you must complete:
■ Create the Salary class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the calculated amount, scope/access result required by the question.

Problem 279
Problem: Create a complex order-processing method containing several nested blocks and identify the scope of every variable.
What you must complete:
■ Create the Order class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Keep method-local variables/parameters inside their declaring method.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 280
Problem: Create nested try, catch, and finally blocks and analyze variable accessibility.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the exception-handling variables inside their specified blocks and test valid/invalid access points.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 281
Problem: Create nested loops with shadowed variables and predict output.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Create the required shadowing and resolve the instance field with this.field where needed.
■ Declare the loop variable inside the loop and test its visibility before, during, and after the loop.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the execution order/output, scope/access result required by the question.

Problem 282
Problem: Create static field → instance field → parameter → local variable with the same name. Determine which variable is accessed at each point.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Create the required shadowing and resolve the instance field with this.field where needed.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the comparison result, scope/access result required by the question.

Problem 283
Problem: Fix a realistic banking program containing several scope-related compilation errors.
What you must complete:
■ Create the Bank class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Identify and fix every use of a variable outside its legal scope.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 284
Problem: Create a customer-management program containing multiple levels of variable shadowing and fix it using this.
What you must complete:
■ Create the Customer class.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Create the required shadowing and resolve the instance field with this.field where needed.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Problem 285
Problem: Analyze a 50-line real-life program and identify the scope of every variable without executing it.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ Declare each variable at the exact scope the problem asks you to demonstrate.
■ Declare the conditional variable inside the if/else block and test access inside and outside that block.
■ Mark which declaration is selected at each variable use.
■ Print the requested values or record the expected compilation error where the problem is about invalid scope.
■ Desired output: clearly show the scope/access result required by the question.

Stack & Heap

Problem 286
Problem: Create Bank → Customer → BankAccount → Transaction and draw the conceptual heap and references.
What you must complete:
■ Create the required classes: BankAccount, Customer, Transaction, Bank.
■ List every local/parameter reference and every object created with new.

■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the count/number, final reference/object state required by the question.

Problem 287
Problem: Trace stack frames when a UPI payment passes through main() → pay() → validate() → transfer().
What you must complete:
■ Create the required classes: Payment, UPI.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the execution order/output, final reference/object state required by the question.

Problem 288
Problem: Draw the stack and heap for Customer → Cart → Product.
What you must complete:
■ Create the required classes: Product, Car, Customer, Cart.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 289
Problem: Create an order containing several product objects and trace all references.
What you must complete:
■ Create the required classes: Product, Order.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the execution order/output, final reference/object state required by the question.

Problem 290
Problem: Trace passenger, train, seat, and ticket references during railway booking.
What you must complete:
■ Create the required classes: Book, Passenger, Booking, Train, Seat, Ticket.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the execution order/output, final reference/object state required by the question.

Problem 291
Problem: Create patient and appointment objects and draw their reference relationships.
What you must complete:
■ Create the required classes: Patient, Appointment.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 292
Problem: Create two objects that reference each other and determine whether they can become garbage-collection candidates.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ After each reference is removed/null, check whether any other live reference can still reach the object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 293
Problem: Create a static reference to a shared object and determine why it remains reachable.

What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Include the static reference as a class-level/root reference when checking reachability.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 294
Problem: Pass an object to a method, reassign the parameter, and determine whether the caller's reference changes.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Show the parameter/reference changing targets and separately show whether the caller's reference changes.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 295
Problem: Pass an object to a method and modify its field. Explain the difference between mutation and reassignment.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Show the parameter/reference changing targets and separately show whether the caller's reference changes.
■ Show the object field changing while the reference still points to the same heap object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 296
Problem: Create a recursive method that creates an object at each call. Trace stack growth and heap allocation.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Draw one stack frame for each active recursive call and one heap object for each required allocation.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the execution order/output, final reference/object state required by the question.

Problem 297
Problem: Create Company → Department → Employee → Address and draw the object graph.
What you must complete:
■ Create the required classes: Employee, Department, Company, Address.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 298
Problem: Create multiple references to several objects and determine exactly which objects become unreachable after each reference is removed.
What you must complete:
■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ After each reference is removed/null, check whether any other live reference can still reach the object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 299

Problem: Given a 40–50 line Java program, identify stack frames, local variables, references, heap objects, static references, object relationships, and unreachable objects.

What you must complete:

■ Create the class/classes named or implied by the problem.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Include the static reference as a class-level/root reference when checking reachability.
■ After each reference is removed/null, check whether any other live reference can still reach the object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the final reference/object state required by the question.

Problem 300

Problem: FINAL MASTER PROJECT: Build a banking application with Bank, Customer, BankAccount, Transaction, and BankConfiguration. Use all seven topics: classes/objects, this, static, constructors, initialization blocks, scope, and stack/heap analysis. Support registration, IDs, accounts, deposits, withdrawals, transfers, transactions, balance checks, multiple customers/accounts, transaction counting, bank-wide interest, constructor overloading/chaining, method chaining where appropriate, static/instance initialization, proper scope, reference tracing, and garbage-collection analysis.

What you must complete:

■ Create the required classes: BankAccount, Customer, Transaction, Bank, BankConfiguration.
■ List every local/parameter reference and every object created with new.
■ Draw each object separately in the conceptual heap and each reference separately in the relevant stack frame.
■ Create a stack frame for each active method call and place its parameters/local variables there.
■ Include the static reference as a class-level/root reference when checking reachability.
■ After each reference is removed/null, check whether any other live reference can still reach the object.
■ Update the diagram after each important statement.
■ State the final reachable/unreachable objects and the requested output/state.
■ Desired output: clearly show the count/number, scope/access result, final reference/object state required by the question.

FINAL SELF-CHECK

■ Every required class has been created.

■ Every required field has been declared with a suitable type.

■ Every required constructor/method/block has been implemented.

■ The required objects have actually been created.

■ The requested operations/calculations have been performed.

■ The output clearly shows the result requested by the problem.

■ Any comparison/search/count/average result is visible.

■ Scope problems have been checked for valid/invalid access.

■ Stack/heap problems include references, objects, and reachability.

■ The final program has been tested with sensible sample data.